Analyse discriminante linéaire prédictive : un package R6 et une application Shiny

Package `ldaR6` — M2 SISE, Programmation R, 2026–2027

MAMPOUYA CLARK Chancy Loïc Franlly

Travail individuel

Rendu du 15 novembre 2026

Résumé

Je présente `ldaR6`, un package R d'analyse discriminante linéaire (ADL) prédictive dont le cœur est implémenté *from scratch* avec une classe R6. Il propose les méthodes `fit`, `predict` et `predict_proba`, les fonctions de classement, le lambda de Wilks, des tests de pertinence et la sélection pas-à-pas des variables. Les variables catégorielles sont recodées et les calculs optimisés pour de grands volumes. Une application Shiny permet de charger les données, visualiser les résultats et prédire une classe individuellement ou par fichier. Une documentation en anglais, des tests unitaires et des benchmarks comparatifs avec `MASS::lda` complètent le projet.

Table des matières

Résumé ii

1 Cahier des charges et démarche 1

2 Fondements théoriques 1

2.1 Modèle et règle de décision . . . 1
2.2 Axes discriminants . . . 1
2.3 Lambda de Wilks et tests . . . 1

3 Architecture du programme 2

3.1 Modules R . . . 2
3.2 Interface de la classe . . . 2
3.3 Choix d'architecture : tout intégré plutôt qu'un pipeline . . . 3

4 Choix méthodologiques 3

4.1 Variables explicatives catégorielles . . . 3
4.2 Sélection de variables et analyse des contributions . . . 3
4.3 Singularité, grande dimension et valeurs manquantes . . . 4

5 Implémentation et performances 4

5.1 Algorithme de `fit` . . . 4
5.2 Banc d'essai . . . 5
5.3 Robustesse . . . 6

6 Validation des calculs 6

7 Application Shiny 7

8 Livrables, installation et limites 7

A Exemple de sortie de `$summary()` (iris) 7

B Utilisation minimale 8

C Contenu de la suite de tests 8

Références 9

1 Cahier des charges et démarche

Le sujet officiel [1] demande un package R installáble depuis GitHub, dont le cœur de l'ADL est programmé par l'auteur selon la norme R6, avec : un constructeur, `$fit(X, y)`, `$predict(X)`, `$predict_proba(X)`, `$print()`, `$summary()` et des propriétés exposées ; les coefficients du modèle et des indicateurs d'évaluation globale et de pertinence des variables ; une stratégie pour les explicatives catégorielles et une pour la sélection de variables ; une application Shiny (chargement, choix des variables explicatives et illustratives, calculs, prédiction individuelle et par fichier) ; un rapport LaTeX (≤ 12 pages), une aide et un tutoriel en anglais, une archive de secours. La rapidité et la prise en compte des grandes volumétries sont des critères d'évaluation. Le lambda de Wilks et les tests associés correspondent à des choix d'implémentation pour l'interprétation statistique. L'ouvrage de référence de Ricco Rakotomalala [2] guide la présentation statistique.

Ma démarche a été : (i) formuler l'algorithme à partir de *statistiques suffisantes* (moyennes par classe et matrices de sommes de carrés et de produits croisés, SSCP) ; (ii) faire reposer tous les tests et la sélection sur ces seules matrices $p \times p$, de sorte que leur coût ne dépende plus de n ; (iii) valider chaque brique contre des références indépendantes (`MASS::lda`, `stats::manova`, identités mathématiques de l'ADL) ; (iv) mesurer les performances et chasser les défaillances.

2 Fondements théoriques

2.1 Modèle et règle de décision

Soit n individus décrits par p variables numériques $\mathbf{x} \in \mathbb{R}^p$ et répartis en K classes d'effectifs n_k. On note $\mathbf{m}_k$ le centre de la classe k, $\mathbf{m}$ le centre global et

$$\mathbf{W} = \sum_{k=1}^{K} \sum_{i \in k} (\mathbf{x}_i - \mathbf{m}_k)(\mathbf{x}_i - \mathbf{m}_k)^\top, \quad \mathbf{B} = \sum_{k=1}^{K} n_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^\top, \quad \mathbf{T} = \mathbf{W} + \mathbf{B}. \tag{1}$$

Sous l'hypothèse gaussienne à covariance commune $\boldsymbol{\Sigma}$ estimée par $\hat{\boldsymbol{\Sigma}} = \mathbf{W}/(n-K)$, la règle de Bayes est linéaire : la *fonction de classement* de la classe k est

$$f_k(\mathbf{x}) = \ln \pi_k - \tfrac{1}{2} \mathbf{m}_k^\top \hat{\boldsymbol{\Sigma}}^{-1} \mathbf{m}_k + (\hat{\boldsymbol{\Sigma}}^{-1} \mathbf{m}_k)^\top \mathbf{x}, \tag{2}$$

avec π_k la probabilité a priori (proportionnelle aux effectifs, égale, ou fournie). L'individu est affecté à $\arg\max_k f_k$ et les probabilités a posteriori sont $P(k|\mathbf{x}) = e^{f_k} / \sum_j e^{f_j}$ (calculées après soustraction du maximum pour la stabilité numérique). Les coefficients du score de classement sont les composantes de $\hat{\boldsymbol{\Sigma}}^{-1}\mathbf{m}_k$; la constante intègre la probabilité a priori par le terme $\ln \pi_k$.

2.2 Axes discriminants

Les axes canoniques résolvent $\mathbf{B}\mathbf{a} = \ell \hat{\boldsymbol{\Sigma}}\mathbf{a}$. Comme $\mathbf{B} = \mathbf{D}\mathbf{D}^\top$ avec $\mathbf{D} = [\sqrt{n_k}(\mathbf{m}_k - \mathbf{m})]_k$ de rang $\leq K-1$, le problème se ramène à la matrice $K \times K$ $\mathbf{C} = \mathbf{D}^\top \hat{\boldsymbol{\Sigma}}^{-1} \mathbf{D}$: si $\mathbf{C}\mathbf{v} = \ell \mathbf{v}$, alors $\mathbf{a} = \hat{\boldsymbol{\Sigma}}^{-1}\mathbf{D}\mathbf{v}/\sqrt{\ell}$ vérifie $\mathbf{a}^\top \hat{\boldsymbol{\Sigma}} \mathbf{a} = 1$. On évite ainsi toute décomposition propre $p \times p$ (voir §5). Les valeurs propres $\lambda_j = \ell_j/(n-K)$ de $\mathbf{W}^{-1}\mathbf{B}$ donnent les corrélations canoniques $\sqrt{\lambda_j/(1+\lambda_j)}$.

2.3 Lambda de Wilks et tests

Le lambda de Wilks mesure le pouvoir discriminant global :

$$\Lambda = \frac{|\mathbf{W}|}{|\mathbf{T}|} = \prod_j \frac{1}{1+\lambda_j} \in [0,1] \quad \text{(petit = bon)}. \tag{3}$$

Pour q variables, $h = K-1$ degrés de liberté d'hypothèse et e d'erreur, l'approximation de Rao donne $F = \frac{1-\Lambda^{1/t}}{\Lambda^{1/t}} \cdot \frac{df_2}{df_1}$ avec $df_1 = qh$, $df_2 = wt - \frac{qh-2}{2}$, $w = e + h - \frac{q+h+1}{2}$ et $t = \sqrt{\frac{q^2h^2-4}{q^2+h^2-5}}$ ($t = 1$ si le dénominateur est ≤ 0) ; elle est exacte pour $q = 1$ ou $h \leq 2$. Le package fournit aussi les traces de Pillai et de Hotelling–Lawley, la plus grande racine de Roy et le χ^2 de Bartlett (global puis par axe).

Test partiel. L'apport d'une variable (ou d'un bloc B de colonnes) j *sachant* les autres est $\Lambda(j|S) = \Lambda(S \cup j)/\Lambda(S)$; il se lit directement sur les inverses des SSCP du modèle courant :

$$\Lambda(B \,|\, S \setminus B) = \frac{|(\mathbf{T}_{SS}^{-1})_{BB}|}{|(\mathbf{W}_{SS}^{-1})_{BB}|}, \tag{4}$$

testé par la loi de Rao avec $q = |B|$ et $e = n - K - (|S| - q)$. Pour les candidats à l'entrée, on utilise les SSCP résiduelles après régression sur S, calculées pour tous les candidats d'un coup (opérations matricielles, §4.2). Enfin, les distances de Mahalanobis $D^2_{kl} = (\mathbf{m}_k - \mathbf{m}_l)^\top \hat{\mathbf{\Sigma}}^{-1} (\mathbf{m}_k - \mathbf{m}_l)$ sont testées par $F = \frac{(n-K-p+1)\, n_k n_l}{p(n-K)(n_k+n_l)} D^2_{kl}$.

3 Architecture du programme

3.1 Modules R

Fichier	Rôle
`R/LDA.R`	classe R6 `LDA` : `fit`, `predict`, `predict_proba`, `decision_function`, `transform`, `score`, `confusion`, `cross_validate`, `variable_audit`, `project_illustrative`, `print`, `summary`, `plot` ; propriétés en lecture seule (liaisons actives)
`R/encoding.R`	apprentissage (`.make_encoder`) et application (`.apply_encoder`) du codage des catégorielles
`R/stats_utils.R`	log-déterminant, inverses (Cholesky, pseudo-inverse), F de Rao, statistiques partielles d'entrée et de sortie
`R/selection.R`	sélection pas-à-pas (forward, backward, both) sur les SSCP
`R/utils_public.R`	`read_data`, `simulate_lda_data`, `run_app`, méthodes S3 (`predict`, `coef`, `summary`, `plot`)
`inst/shiny/app/app.R`	application Shiny

Les fonctions internes sont pures (SSCP en entrée, statistiques en sortie), donc testables sans données brutes ; la classe orchestre.

3.2 Interface de la classe

`LDA$new(priors, encoding, selection, alpha_enter, alpha_stay, max_vars, shrinkage, na_action, chunk_size)` ne fait que stocker les paramètres. `$fit(X, y)` renvoie l'objet (chaînage) et remplit un enregistrement privé exposé par des liaisons actives. Les principaux objets générés sont :

`$coefficients`	matrice $(p+1) \times K$ des fonctions de classement (constante en première ligne)
`$wilks`	`global` (lambda, F, df_1, df_2, p), `multivariate` (Wilks, Pillai, Hotelling–Lawley, Roy), Bartlett
`$variables`	par variable : lambda univarié et partiel, F, p, `status` $\in$ {contributing, redundant, uninformative}
`$canonical`	coefficients des axes, valeurs propres, corrélations canoniques et tests par axe, structure, centroïdes
`$distances`	D^2 entre classes et p-values
`$selected`, `$selection_history`	variables retenues ; chemin (étape, action, lambda partiel, F, p, Wilks cumulé)
`$confusion_train`, `$accuracy_train`, `$scores_train`	résultats en resubstitution et coordonnées factorielles
`$means`, `$pooled_cov`, `$priors`, `$classes`, `$features`	ingrédients du modèle et description des variables

Les génériques R usuels sont aussi disponibles (`predict(m, newdata, type="prob")`, `coef`, `summary`, `plot`).

3.3 Choix d'architecture : tout intégré plutôt qu'un pipeline

Le sujet autorise soit l'intégration des traitements (catégorielles, sélection) dans le modèle, soit un pipeline séparé façon scikit-learn. J'ai choisi l'**intégration** dans la classe, avec des modules internes distincts, pour quatre raisons :

1. *Pas de fuite d'information* : l'encodeur (niveaux, colonnes) et la sélection sont appris sur l'échantillon d'apprentissage puis rejoués à l'identique par `predict`. La validation croisée refait tout le pipeline dans chaque pli (`cross_validate`), ce qui évite l'optimisme classique d'une sélection faite sur toutes les données.
2. *Performance* : la sélection réutilise les SSCP $\mathbf{W}$ et $\mathbf{T}$ déjà calculées pour le modèle ; un pipeline séparé les recalculerait ou imposerait de les faire circuler.
3. *Prédiction minimale* : après sélection, l'encodeur est restreint aux colonnes utiles, donc `predict` n'exige que les variables retenues (utile pour le formulaire individuel de Shiny).
4. *Utilisabilité* : un seul objet, un seul contrat `fit/predict`.

Contrepartie assumée : le critère de sélection (Wilks) est lié à l'ADL et ne se découple pas ; la maintenance exige de respecter la frontière entre fonctions pures et classe, appliquée fichier par fichier.

4 Choix méthodologiques

4.1 Variables explicatives catégorielles

Le calcul matriciel de l'ADL exige des colonnes numériques. Le codage est effectué *en amont*, sans toucher à l'algorithme, avec trois options (paramètre `encoding`) :

— **dummy** (défaut) : $k-1$ indicatrices, la première modalité servant de référence. C'est le choix qui conserve $\hat{\mathbf{\Sigma}}$ inversible ; chaque coefficient s'interprète par rapport à la modalité de référence.

— **one-hot** : k indicatrices. Leur somme vaut 1 pour tout individu, donc $\hat{\mathbf{\Sigma}}$ est singulière (colinéarité parfaite). Le package bascule alors sur une pseudo-inverse de Moore–Penrose et émet un avertissement. Un test unitaire a vérifié que les prédictions sont strictement identiques à celles du codage dummy : les deux paramétrisations engendrent le même espace, seul l'aspect des coefficients change.

— **label** : un entier par variable. Compact mais il impose un ordre et des distances entre modalités : acceptable seulement pour des facteurs ordonnés, trompeur sinon.

Conséquences. Des indicatrices ne sont pas gaussiennes : l'ADL reste un classifieur linéaire raisonnable (pratique courante) mais les tests de Wilks deviennent approximatifs pour les facteurs très déséquilibrés. Une modalité présente dans une seule classe sans variance interne produit une colonne dégénérée, retirée avec avertissement. Lors de la sélection, une catégorielle entre ou sort *comme bloc*, testée par un lambda partiel multivarié ($q = k-1$) : on évite de garder une modalité isolée. Une alternative que je n'ai pas implémentée est DISQUAL (ACM puis ADL, [7]).

4.2 Sélection de variables et analyse des contributions

Critère : le lambda de Wilks partiel et son test F de Rao (seuils `alpha_enter`, `alpha_stay`, défaut 0,05). Trois stratégies : *forward*, *backward* et *both* (pas-à-pas).

```
SELECT(W, T, blocs, direction, a_in, a_out):
  S <- {} (forward/both)  ou  tous les blocs (backward)
  repeter (garde-fou : 4*nb_blocs + 5 tours)
    si direction in {backward, both}:                # sortie
        pour chaque bloc B de S : p_B <- test(Lambda(B | S\B))   # via inverses de W_SS, T_SS
        si max p_B > a_out : retirer ce bloc ; (continuer si backward)
    si direction in {forward, both} et |S| < max_vars:  # entree
        pour tous les candidats c hors de S : Lambda(c | S) <- det(W_res)/det(T_res)
           avec W_res = W_cc - W_cS W_SS^-1 W_Sc   (vectorise pour les blocs de taille 1)
        c* <- candidat de plus petite p-value ; si p <= a_in : ajouter c*
  jusqu'a stabilite ;  retourner S et l'historique (etape, action, Lambda, F, p, Wilks)
```

Chaque étape coûte $O(|S|^2 p)$ pour tous les candidats simultanément, indépendamment de n. Pour *toutes* les variables, y compris les non retenues, `$variables` donne le test univarié (variable seule) et le test partiel (sachant le modèle final). Le package en déduit le `status` demandé par le sujet : *contributing* (apporte de l'information sachant les autres), *redundant* (informative seule mais sans apport additionnel : par exemple

`income_copy`, copie bruitée de `income`) et *uninformative* (aucun lien avec la cible). Les variables *perturbatrices* sont détectées par `variable_audit` : une variable est perturbatrice si son retrait *augmente* la précision en validation croisée (mêmes plis, même graine). La Figure 1 montre le résultat sur les données démo : les cinq variables numériques utiles, `housing` et `education` sont retenues ; `income_copy` (redondante), `noise1`, `noise2` et `job` sont écartées. **Limite** : les p-values après sélection sont optimistes ; seule la validation croisée incluant la sélection donne une précision honnête.

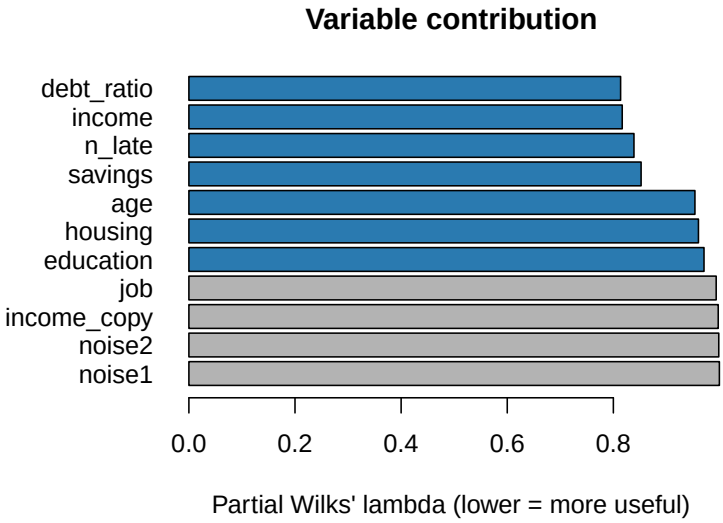


FIGURE 1 – Lambda de Wilks partiel par variable (`credit_demo`, sélection pas-à-pas) ; en bleu les variables apportant de l'information sachant les autres.

4.3 Singularité, grande dimension et valeurs manquantes

Lorsque $\hat{\mathbf{\Sigma}}$ est singulière ($p \geq n - K$, colinéarité), l'inversion de Cholesky est remplacée par une pseudo-inverse spectrale avec avertissement explicite. L'option `shrinkage`$= \gamma$ remplace $\hat{\mathbf{\Sigma}}$ par $(1-\gamma)\hat{\mathbf{\Sigma}} + \gamma\,\mathrm{diag}(\hat{\mathbf{\Sigma}})$ (rétrécissement vers la diagonale, [5, 6]) : toujours inversible, *invariant d'échelle*. La première version rétrécissait vers $\bar{s}\,\mathbf{I}$; un test a montré qu'une variable de grande variance (`savings`) dominait alors le terme et détériorait la précision (69 % au lieu de 82 %) ; ce choix a été corrigé. Les tests partiels sont désactivés (`NA`) si $\mathbf{W}_{SS}$ est singulière, car ils n'ont alors pas de sens. Les lignes incomplètes sont retirées avec avertissement (ou erreur selon `na_action`) ; en prédiction, une ligne incomplète ou à modalité inconnue reçoit `NA`.

5 Implémentation et performances

5.1 Algorithme de `fit`

```
FIT(X, y):
  1 valider ; retirer NA ; apprendre encodeur ; M <- encoder(X)     # O(np)
  2 moyennes <- rowsum(M, y) ; B <- crossprod(sqrt(n_k)*(m_k-m))
  3 W <- 0 ; pour chaque bloc de chunk_size lignes :
      W += crossprod(M_bloc - m[y_bloc])                             # O(np^2)
  4 retirer colonnes sans variance intra ; T <- W + B
  5 si selection != none : S <- SELECT(W, T, ...)
  6 Sigma <- W_SS/(n-K) ; Sigma <- (1-g)*Sigma + g*diag(Sigma)
      Sinv <- chol2inv(chol(Sigma))  # repli pseudo-inverse si necessaire
  7 coef <- Sinv t(m_S) ; const <- ln(prior) - 0.5 colSums(t(m_S)*coef)
  8 Wilks <- exp(logdet W_SS - logdet T_SS) ; Rao F ; Bartlett ; D^2
      axes via C = D' Sinv D (K x K)
  9 Z <- M_S coef + const ; confusion ; tableau des tests par variable
```

Complexité : $O(np^2 + p^3 + Kp^2)$, avec np^2 dominant pour les données longues. Les points d'optimisation sont : `crossprod` (BLAS) ; `rowsum` (C) pour les centres ; accumulation par blocs de lignes (temporaires limités, résultat identique, test unitaire à `chunk_size=700`) ; axes canoniques par une matrice $K \times K$ plutôt

qu'une décomposition $p \times p$; log-déterminants par Cholesky (pas de dépassement) ; tests partiels vectorisés. La prédiction est une multiplication matricielle $O(npK)$.

5.2 Banc d'essai

Les chiffres comparatifs du tableau et de la figure ci-dessous proviennent du banc d'essai initial (fichier `benchmarks/benchmark_results.csv`) ; ils n'ont pas été reproduits sur la version corrigée. Ils sont conservés comme *résultats historiques* et ne doivent pas être confondus avec les nouvelles mesures locales. Le script `benchmarks/benchmark.R` compare `LDA$fit` à `MASS::lda` sur données gaussiennes simulées (`simulate_lda_data`), temps médian de 1 à 3 répétitions. Pour chaque cas, les classes prédites coïncident dans 100 % des cas avec celles de `MASS::lda` (contrôle sur les 5 000 premières lignes).

Cas	n	p	`MASS::lda` (s)	`ldaR6` (s)	gain
long	10^5	10	0,17	0,12	1,4×
long	10^6	10	2,79	1,57	1,8×
long	10^6	50	31,03	9,04	3,4×
intermédiaire	5 000	50	0,14	0,13	1,1×
intermédiaire	20 000	200	5,95	1,40	4,3×

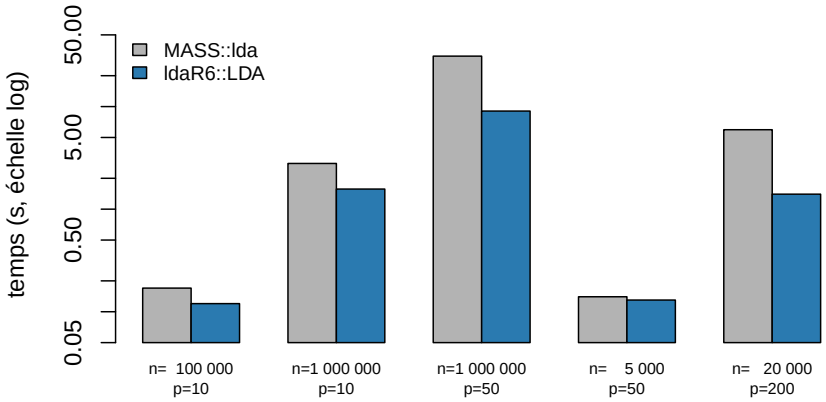


FIGURE 2 – Temps d'ajustement (échelle logarithmique), données longues et intermédiaires.

Données larges ($n = 200$, $K = 3$; `MASS::lda` non comparé car $p > n - K$ rend la covariance singulière) :

p	pseudo-inverse (s)	`shrinkage=0.2` (s)	`selection="forward"` (s)
500	0,58	0,33	0,30
2 000	14,14	4,77	1,17

La pseudo-inverse coûte une décomposition propre $O(p^3)$; le rétrécissement la remplace par Cholesky (3× plus rapide à $p = 2000$) et la sélection pas-à-pas, qui n'inverse que de petites matrices, est la plus rapide. Un autre essai a revélé un gain de ×13 après avoir supprimé deux décompositions propres $p \times p$ (22 s → 1,7 s pour $p = 1000$). Pour $p \gg n$, la précision d'apprentissage d'une pseudo-inverse n'a pas de valeur prédictive (0,555 pour $p = 2\,000$ alors que seules 5 variables informent) : on utilise `cross_validate` pour juger, la sélection ou le rétrécissement pour fiabiliser. Validation croisée 10 plis ($n = 2 \cdot 10^4$, $p = 50$) : 1,5 s ; sélection pas-à-pas : 0,9 s.

Mesures locales complémentaires (novembre 2026). Sur la version testée sous R 4.4.2 (Windows), trois appels distincts à une fonction de mesure basée sur `system.time` ont fourni les temps *elapsed* suivants :

n (apprentissage)	p	n_{pred}	fit (s)	predict (s)
10 000	10	1 000	0,06	0,00
100 000	10	10 000	0,45	0,01
100 000	10	100 000	0,28	0,05

Ces temps sont ponctuels, arrondis à 0,01 s ; `0,00` n'est pas un temps nul. Ils ne mesurent ni la mémoire maximale ni un gain face à `MASS::lda` sur cette machine. Une comparaison reproductible nécessite plusieurs répétitions et la même configuration de données.

Mémoire et reproductibilité. Le traitement par blocs ne supprime pas la matrice encodée complète : la mémoire est de l'ordre de $O(np + p^2)$, avec plusieurs matrices $p \times p$. Une seule matrice dense de $10\,000 \times 10\,000$ doubles occupe environ 800 Mo. Les temps d'apprentissage et de prédiction doivent être mesurés séparément ; le script conserve désormais les versions et les informations matérielles disponibles. La taille de l'objet R ne mesure pas le pic mémoire du processus.

5.3 Robustesse

Le banc d'essai a servi de test de charge : à $n = 10^6$ il a révélé un dépassement d'entier dans le test de distance ($n_k n_l$ calculé en entiers), corrigé en stockant les effectifs en réels. Les cas limites suivants sont couverts par des tests : colonne constante (retirée, avertissement), facteur à un niveau, classe à une seule observation, deux classes, $p > n$, NA en apprentissage et en prédiction, modalité inconnue, colonne manquante (erreur explicite), objet non ajusté (erreur explicite), invariance à l'échelle et au décalage d'une variable.

6 Validation des calculs

Références et tests fournis. Les comparaisons numériques de précision qui suivent proviennent de la suite initiale ; une exécution séparée de tous les tests `testthat` reste à documenter pour la version corrigée. Sur `iris` : probabilités a posteriori identiques à `MASS::lda` à $5 \cdot 10^{-15}$ près, coefficients des axes à 10^{-14} ; lambda de Wilks, F de Rao et trace de Pillai identiques à `stats::manova` ($\Lambda = 0{,}02344$, $F(8, 288) = 199{,}1$) ; lambda partiel confronté à sa définition (modèle complet / réduit) ; fonctions de classement recalculées ligne à ligne à partir de l'équation (2). Les coefficients de `Sepal.Length` (23,54417 ; 15,69821 ; 12,44585) sont reproduits dans l'exemple numérique de l'annexe A.

Portée de la validation. Les comparaisons numériques portent sur les fonctions implémentées en R et les références `MASS::lda` et `stats::manova`. Les résultats de la suite initiale sont distingués des mesures locales récentes.

Exemple de résultats. Le test initial de `credit_demo` ($n = 900$, 3 classes) avec 11 variables candidates sélectionne 7 variables (dont `education`) : $\Lambda = 0{,}232$, précision 82,6 % en resubstitution et 81,6 % en validation croisée 10 plis. Dans la démonstration locale récente, avec seulement 7 variables candidates (sans `education`), la configuration sans sélection atteint 81,6 % (entraînement et validation croisée 5 plis) ; la sélection `both` retire `job`, conserve 6/7 variables et obtient 81,8 % à l'entraînement, 81,0 % en validation croisée 5 plis ($\Lambda = 0{,}239$). Les ensembles de variables et les nombres de plis diffèrent : ces résultats ne sont pas directement comparables. Sur `iris`, précision 98 % en entraînement et validation croisée (Figure 3).

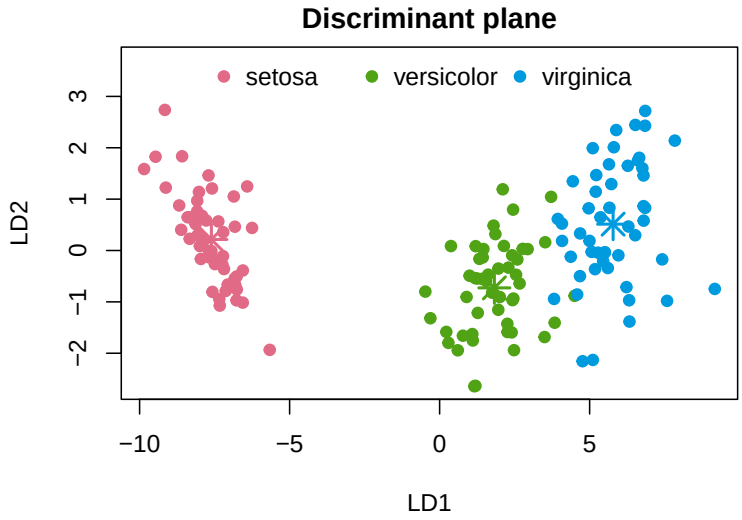


FIGURE 3 – Plan discriminant (`iris`) ; les étoiles sont les centroïdes.

7 Application Shiny

`ldaR6::run_app()` lance une application à quatre étapes pensée pour un débutant : (1) *Données* : CSV/TSV (tabulation par défaut), Excel ou exemples intégrés, aperçu et diagnostic des types ; (2) *Paramétrage* : variable cible, variables explicatives et *illustratives*, a priori, codage, sélection, seuils, rétrécissement, part d'apprentissage et graine, nombre de plis ; (3) *Résultats* : indicateurs clés (précisions d'apprentissage, de test et de validation croisée, Wilks), fonctions de classement téléchargeables, tests, tableau coloré du statut des variables et diagramme des contributions, matrices de confusion, plan discriminant avec individus de test, description des variables illustratives (corrélations, barycentres) et sortie `summary` complète ; (4) *Prédiction* : formulaire généré dynamiquement avec les seules variables du modèle (numérique ou liste de modalités) et histogramme des probabilités, ou *prédiction par fichier* (descripteurs seuls) avec export TSV des classes et des probabilités. Les erreurs et avertissements du package deviennent des notifications lisibles. Les essais manuels dans RStudio confirment en local l'ajustement sur `iris` et `credit_demo`, les sorties statistiques, la prédiction individuelle et la prédiction de trois individus depuis un CSV (`setosa`, `versicolor`, `virginica`). Aucun test automatisé `shiny::testServer` n'est revendiqué dans cette livraison ; l'hébergement en ligne n'est pas nécessaire au sujet.

8 Livrables, installation et limites

- **GitHub** : code, aide, jeux de données (`credit_demo`, `iris`, fichier de nouveaux individus), tutoriel `TUTORIAL.md` (reproductible, en anglais), banc d'essai. D'epôt prévu sous `clark94/ldaR6` ; installation *après publication* : `remotes::install_github("clark94/ldaR6")`. Sa disponibilité en ligne reste à vérifier avant remise.
- **Plan B** : `ldaR6_1.0.0.tar.gz` (`install.packages(..., repos=NULL, type="source")`).
- **Qualité** : `devtools::check(cran = FALSE)` exécuté localement sur R 4.4.2 : **0 erreur, 0 avertissement, 1 note** (`unable to verify current time`, contrôle des horodatages), durée 42,1 s. Aide au format Rd (`?LDA`) et tests `testthat` présents ; leur exécution séparée reste à confirmer.

Limites. (i) Hypothèses gaussienne et d'homoscédasticité (pas de QDA) ; (ii) la pseudo-inverse reste en $O(p^3)$ pour $p \gtrsim 2000$; (iii) pas de leave-one-out en forme fermée ni de parallélisation (la validation croisée est séquentielle) ; (iv) p-values post-sélection optimistes. **Perspectives** : DISQUAL pour les catégorielles, ADL régularisée (lasso), parallélisation des plis, hébergement de l'application.

A Exemple de sortie de `$summary()` (iris)

```
=== Linear Discriminant Analysis: summary ===
n = 150 | K = 3 | columns used = 4 | pooled df = 147
```

```
Classification functions (constant in first row)
               setosa versicolor  virginica
constant     -86.30847  -72.85261 -104.36832
Sepal.Length  23.54417   15.69821   12.44585
Sepal.Width   23.58787    7.07251    3.68528
Petal.Length -16.43064    5.21145   12.76654
Petal.Width  -17.39841    6.43423   21.07911

Global tests
     statistic   value     F df1 df2    p_value
 Wilks' lambda 0.02344 199.1   8 288 1.365e-112
Bartlett chi2 = 546.1 (df = 8), p < 2.2e-16

Canonical axes
 axis eigenvalue proportion canonical_corr   wilks   chi2 df    p_value
  LD1    32.1919   0.991213         0.9848 0.02344 546.12  8 8.871e-113
  LD2     0.2854   0.008787         0.4712 0.77797  36.53  3  5.786e-08

Confusion matrix (resubstitution)      Accuracy: 98.00%
           Predicted
Observed     setosa versicolor virginica
  setosa         50          0         0
  versicolor      0         48         2
  virginica       0          1        49
```

B Utilisation minimale

```
library(ldaR6)
X <- iris[, 1:4]; y <- iris$Species
m <- LDA$new(selection = "none", encoding = "dummy")
m$fit(X, y)                              # apprentissage
m$summary()                               # sorties detaillees
m$predict(X[1:3, , drop=FALSE])           # trois classes
m$predict_proba(X[1:3, , drop=FALSE])     # probabilites
m$cross_validate(X, y, k = 5, seed = 1) # validation croisee
ldaR6::run_app()                          # application Shiny
```

C Contenu de la suite de tests

`test-lda-core.R` : équivalence avec `MASS::lda`, formule de classement, `manova`, lambda partiel, deux classes, API de prédiction, validation des entrées. `test-categorical-selection.R` : codages, modalités inconnues, sélections forward/backward/both, `max_vars`, prédiction sur variables retenues, validation croisée reproductible, variables illustratives. `test-robustness.R` : $p > n$, blocs de lignes, invariance d'échelle, classe singleton, lecture TSV, affichages et graphiques.

Références

[1] Université Lumière Lyon 2, M2 SISE (2026–2027), *Création d'un package pour R : Analyse discriminante prédictive, algorithmes et outils pour l'interprétation des résultats*, sujet de projet de Programmation R (support fourni par l'enseignant).

[2] R. Rakotomalala (2020), *Pratique de l'analyse discriminante linéaire*, version 1.0, Université Lumière Lyon 2. Document PDF consultable : https://eric.univ-lyon2.fr/ricco/cours/cours/Pratique_Analyse_Discriminante_Lineaire.pdf.

[3] S. S. Wilks (1932), Certain generalizations in the analysis of variance, *Biometrika* 24, 471–494. https://doi.org/10.1093/biomet/24.3-4.471

[4] C. R. Rao (1973), *Linear Statistical Inference and its Applications*, Wiley.

[5] J. H. Friedman (1989), Regularized discriminant analysis, *JASA* 84(405), 165–175. https://doi.org/10.1080/01621459.1989.10478752

[6] J. Schäfer, K. Strimmer (2005), A shrinkage approach to large-scale covariance matrix estimation, *Stat. Appl. Genet. Mol. Biol.* 4(1). https://doi.org/10.2202/1544-6115.1175

[7] G. Saporta (2011), *Probabilités, analyse des données et statistique*, Technip (DISQUAL).

[8] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, Springer. https://hastie.su.domains/ElemStatLearn/

[9] W. N. Venables, B. D. Ripley, *Modern Applied Statistics with S* (package `MASS`). https://cran.r-project.org/package=MASS

[10] F. Pedregosa et al. (2011), Scikit-learn : machine learning in Python, *JMLR* 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[11] W. Chang, *R6 : encapsulated classes with reference semantics*. https://r6.r-lib.org/

[12] Posit, *Shiny for R*. https://shiny.posit.co/